UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

ESCUELA SUPERIOR TLAHUELILPAN

Licenciatura en Ingeniería de Software

Lenguajes de Programación

REPORTE DE LA PRÁCTICA No. 8

PROGRAMACIÓN CON PYTHON

Sistema de punto de venta “Mueblería El Roble”

David Jonathan Hernández Mejía
Mónica Cornejo Velazquez
301

Introducción

Python es un lenguaje de programación de propósito general muy poderoso y flexible, a la vez que sencillo y fácil de aprender. Este lenguaje fue creado a principios de los noventa por Guido van Rossum en los Países Bajos. Es software libre, y está implementado en todas las plataformas y sistemas operativos habituales.

Toma características de lenguajes predecesores, incluso, compatibilizando la solución de varios de ellos. Por ejemplo, habilita tres formas de imprimir el valor de una variable: desde el entorno interactivo escribiendo su nombre (como en Basic), usando la función print, con concatenación de elementos (al estilo del write de Pascal) o bien con patrones de formato (al estilo del printf de C).

Python se desarrolla bajo una licencia de Open source o código abierto aprobada por OSI, por lo que se puede usar y distribuir libremente, incluso para uso comercial.

La Python Software Foundation (PSF) es una corporación sin fines de lucro 501 (c) (3) que posee los derechos de propiedad intelectual detrás del lenguaje de programación Python. Administramos las licencias de código abierto para Python versión 2.1 y posteriores.

El Python Package Index (PyPI) o en español significa *Índice de paquetes de Python* alberga miles de módulos de terceros para Python.

Tanto la biblioteca estándar de Python como los módulos aportados por la comunidad permiten infinitas posibilidades.

- Desarrollo web e Internet.
- Acceso a la base de datos.
- GUIs de escritorio.
- Científico y numérico.
- Educación.
- Programación de red.
- Desarrollo de Software y Juegos.

Las características del lenguaje de programación Python se resumen a continuación:

- Es un lenguaje interpretado, **no compilado**, usa *tipado dinámico*, *fuertemente tipado*.
- Es *multiplataforma*, lo cual es ventajoso para hacer ejecutable su código fuente entre varios sistemas operativos.
- Es un lenguaje de programación multiparadigma, el cual soporta varios paradigma de programación como *orientación a objetos*, *estructurada*, programación imperativa y, en menor medida, programación funcional.
- En Python, el formato del código (p. ej., la indentación) es estructural.

En Python tiene varios tipos de datos compuestos estándar disponibles por defecto en el intérprete, como los tipos numéricos, secuencias, mapeos y conjuntos usados para agrupar otros valores.

Para el caso de las estructuras de datos se usan variables y constantes las cuales usan operadores para tratar los tipos de datos estándar.

Objetivo

Objetivo General

Desarrollar una interfaz visual en Python que es permita hacer uso de diversos factores de este paradigma de programación.

Objetivos Específicos

- Programar un sistema similar a un punto de venta sin bases de datos
- Generar las interfaces necesarias que permitan la manipulación de la información de entrada, dentro del sistema y de salida del sistema
- Crear interfaces claras, intuitivas y atractivas para los usuarios.

Desarrollo de la actividad práctica

Mi proyecto en Python

Desarrollar en Python, las interfaces necesarias para realizar un sistema de nombre “tienda” que se relacione con la actividad de una pequeña empresa que vende artículos a clientes.

- La empresa debe almacenar información de cada uno de sus clientes.
- La empresa debe almacenar información de los artículos que vende.
- El negocio debe almacenar datos de las ventas que realizan.

Toda venta genera un ticket o nota de venta con su correspondiente detalle que describe:

- Producto (s) que se vende
- Cantidad
- El precio unitario del producto
- Importe del producto (Cantidad * Precio)

(Similares a los que te dan impresos en las tiendas, solo que este será solo digital)

Al final del ticket incluir:

Subtotal = Sumatoria del importe del producto

Iva = Subtotal * .16

Total = Subtotal + iva

El sistema debe incluir un inicio de sesión con la identificación de los diferentes roles necesarios para la actividad.

Sistema desarrollado

Para cumplir con la actividad se desarrolló un sistema de punto de venta para una mueblería, la **Mueblería El Roble**, utilizando Python y la librería tkinter. A continuación se describe cómo se construyó.

Herramientas utilizadas

Herramienta	Uso dentro del sistema
Python 3	Lenguaje de programación en el que se escribió todo el sistema.
tkinter	Creación de la ventana y los widgets: Label, Entry, Button, Listbox, Scrollbar, Frame, LabelFrame, Text y Toplevel.
tkinter.ttk	Notebook para organizar el sistema en pestañas y Combobox para elegir al cliente en la venta.
tkinter.messagebox	Ventanas emergentes de información, advertencia, error y confirmación.
datetime	Obtener la fecha y hora que se imprime en el ticket.

Almacenamiento de la información

La información se genera desde el programa y se guarda en variables. Se usaron **listas de diccionarios**: cada diccionario representa un registro (un producto, un cliente o una venta) y la lista funciona como una tabla.

Roles y permisos

El sistema cuenta con dos roles. Al iniciar sesión se valida el usuario y la contraseña contra el registro usuarios y, según el rol, se muestran u ocultan pestañas y botones.

Rol	Nombre	Usuario / contraseña	Permisos
Administrador	David Jonathan Hernández Mejía	admin / admin123	Venta, Inventario, Clientes (incluye eliminar) e Historial de ventas.
Vendedor	Jonathan Flores Cordero	vendedor / vendedor123	Venta y Clientes (puede registrar y editar, pero no eliminar).

Diseño de la interfaz

La interfaz se construyó en una sola ventana principal con una franja de título. Las secciones se agrupan con recuadros (LabelFrame) y los elementos se acomodan con grid. El sistema tiene las siguientes pantallas:

- **Inicio de sesión:** pide usuario y contraseña; la contraseña se oculta con asteriscos.
- **Ventana principal:** muestra el nombre y rol del usuario, el botón “Cerrar sesión” y las pestañas del sistema (ttk.Notebook).
- **Pestaña Venta:** catálogo de productos, datos de la venta (cliente, código y cantidad), lista de productos en la venta y sección de cobro con subtotal, IVA y total.

- **Pestaña Inventario:** lista de productos y un panel de “Opciones” para agregar, editar y eliminar productos, tomando como base el ejemplo de la práctica.
- **Pestaña Clientes:** misma organización que el inventario, para registrar, editar y eliminar clientes.
- **Pestaña Historial de ventas:** lista de ventas realizadas, número de ventas, total vendido y botón para volver a ver el ticket de una venta.
- **Ventana del ticket:** ventana secundaria (Toplevel) con el ticket de la venta en formato digital.

Para que las columnas de las listas queden alineadas se usó la fuente monoespaciada Courier New y cadenas con formato (f-strings) que fijan el ancho de cada columna. También se fijaron los colores de letras y cajas de texto con option_add() para que la interfaz se vea igual aunque el sistema operativo esté en modo oscuro.

Funciones principales del programa

Función	Descripción
iniciar_sesion()	Valida usuario y contraseña, guarda la sesión y aplica los permisos del rol.
cerrar_sesion()	Pide confirmación, limpia la venta en curso y regresa a la pantalla de inicio de sesión.
buscar_producto(codigo)	Recorre la lista de productos y regresa el que tenga ese código, o None si no existe.
calcular_totales()	Calcula el subtotal, el IVA y el total de la venta en curso.
seleccionar_del_catalogo(evento)	Al hacer clic en el catálogo, copia el código del producto a la caja “Código”.
agregar_a_venta()	Valida código, cantidad y existencia, y agrega el producto a la venta (si ya estaba, suma la cantidad).
quitar_de_venta() / cancelar_venta()	Quita un producto seleccionado o todos los productos de la venta.
cobrar()	Valida el pago, descuenta existencias, guarda la venta y muestra el ticket.
generar_ticket(venta)	Arma el texto del ticket con el detalle, subtotal, IVA, total, pago y cambio.
mostrar_ticket(venta)	Abre una ventana secundaria con el ticket.
agregar_producto() / guardar_producto() / eliminar_producto()	Alta, edición y baja de productos del inventario.
agregar_cliente() / guardar_cliente() / eliminar_cliente()	Alta, edición y baja de clientes.
ir_a_clientes()	Desde la venta, abre la pestaña Clientes para registrar un cliente nuevo.

Función	Descripción
mostrar_catalogo(), mostrar_inventario(), mostrar_clientes(), mostrar_historial()	Vuelven a llenar las listas de la interfaz cada vez que cambian los datos.

Proceso de venta y cálculo del ticket

Una venta se realiza con los siguientes pasos:

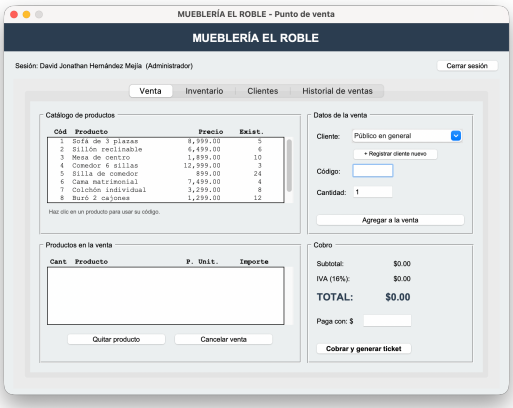
1. El vendedor elige al cliente en la lista desplegable. Si el cliente es nuevo, lo registra con el botón “+ Registrar cliente nuevo” y queda seleccionado automáticamente.
2. Escribe el código del producto (o hace clic en el catálogo) y la cantidad. El sistema valida que el código exista y que haya existencia suficiente.
3. El sistema calcula el **importe** de cada renglón: cantidad × precio unitario.
4. Con cada cambio se recalculan el **Subtotal** (suma de importes), el **IVA** (Subtotal × 0.16) y el **Total** (Subtotal + IVA).
5. El vendedor captura con cuánto paga el cliente; el sistema valida que el pago cubra el total y calcula el cambio.
6. Se descuentan las existencias del inventario, la venta se guarda en la lista ventas y se muestra el ticket.

Resultados

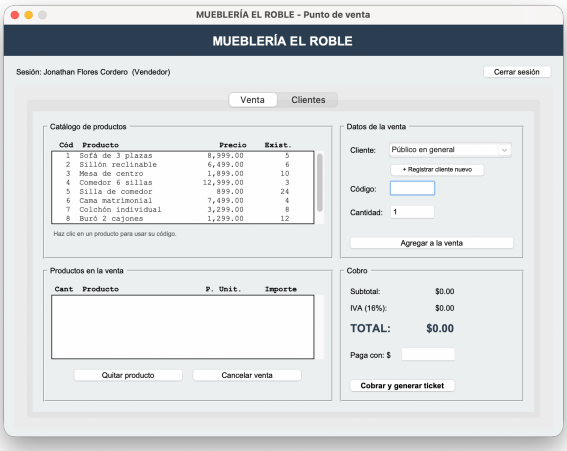
Inicio de Sesión. Si el usuario o la contraseña son incorrectos, el sistema muestra un mensaje de error y no permite el acceso.



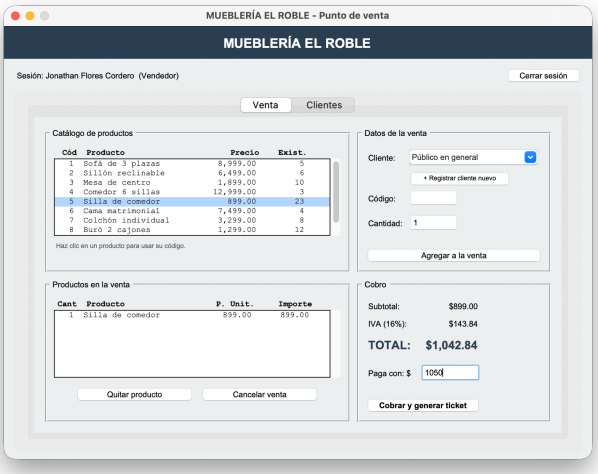
Sesión de admin.



Sesión vendedor

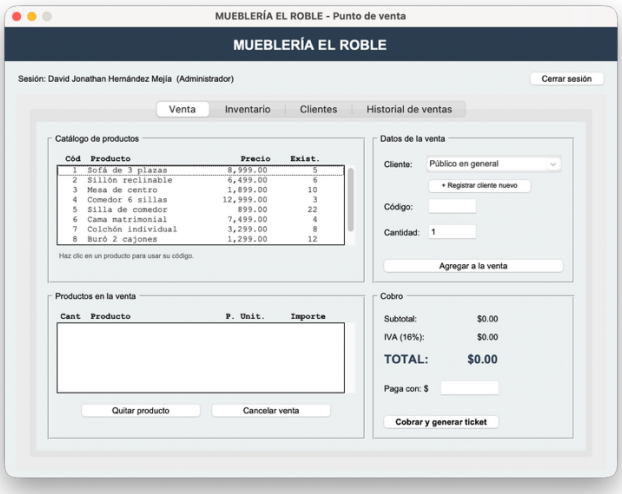


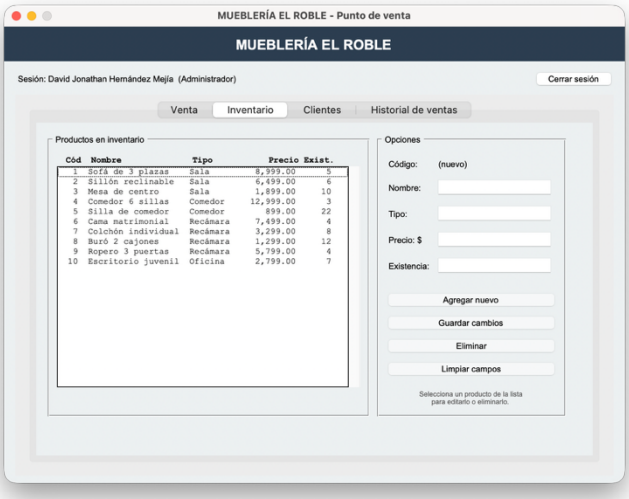
Registro de venta

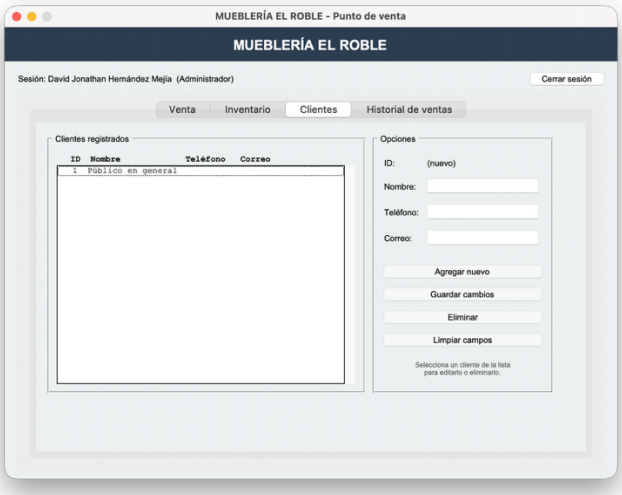


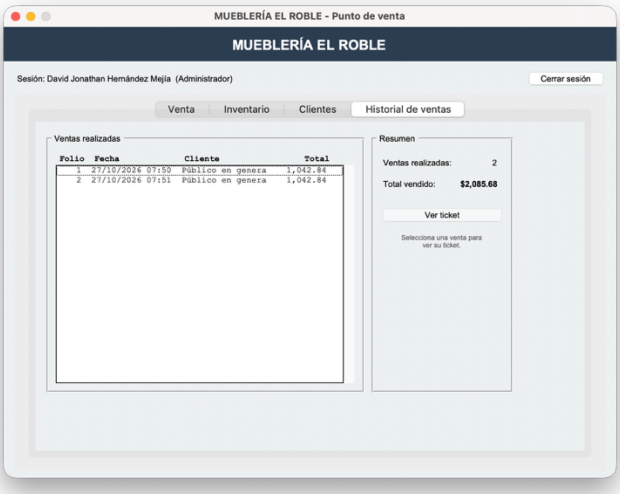


Navegación

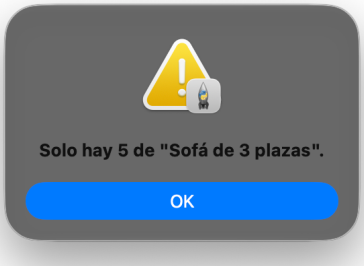




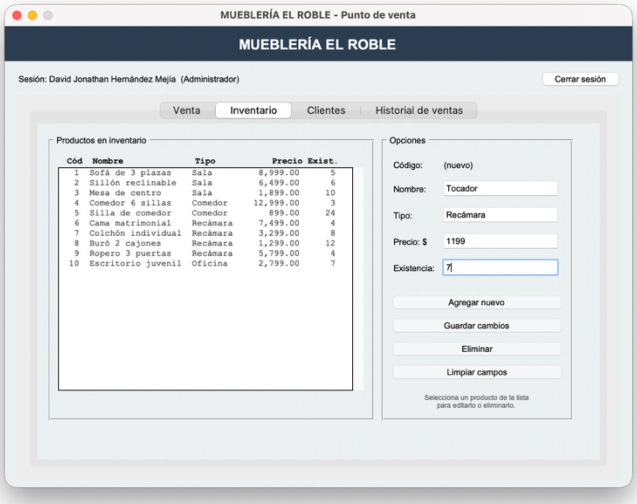




Inventario insuficiente

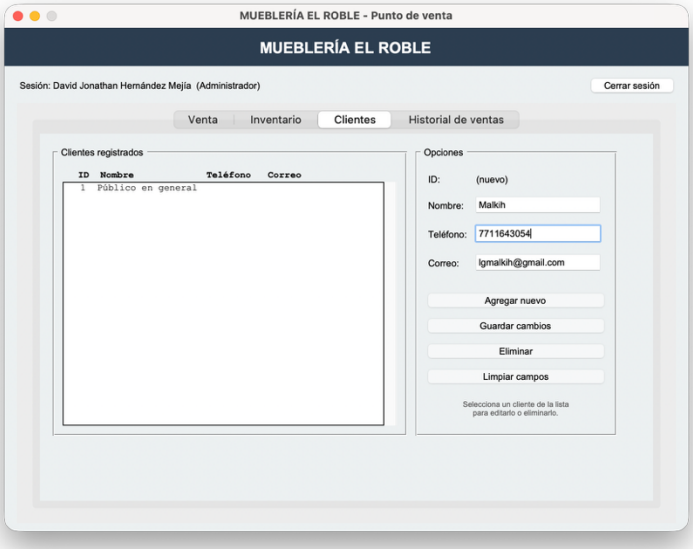


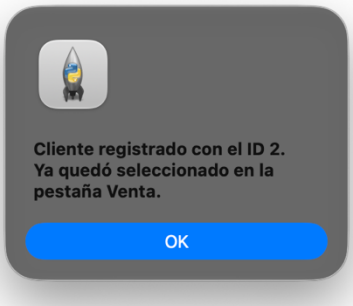
Agregar inventario





Agregar clientes





Discusión

La programación orientada a eventos fue la base del sistema. En un programa de consola, el programador decide el orden en que se pide la información, se procesa y se muestra el resultado. En una interfaz gráfica ese orden lo decide el usuario: puede agregar productos, registrar un cliente, cambiar de pestaña o cobrar en el momento que quiera. Por eso el programa no sigue una secuencia fija de instrucciones. Se queda esperando a que ocurra un evento, como un clic, la selección de un elemento o el cierre de una ventana, y responde ejecutando la función asociada. Este modelo es la base de casi todas las aplicaciones con interfaz gráfica, como los puntos de venta, las aplicaciones móviles y las páginas web, y por eso es importante dominarlo.

En el sistema, este modelo se implementó con tres elementos de tkinter. El primero es el ciclo principal de la ventana, que la mantiene abierta y atenta a las acciones del usuario. El segundo son las funciones que responden a cada evento, como iniciar sesión, agregar un producto a la venta, cobrar o registrar un cliente, que se conectaron a sus botones para ejecutarse cada vez que se da clic. El tercero son los eventos de selección en las listas. Gracias a ellos, al hacer clic en un producto del catálogo su código se copia a la venta, y al seleccionar un producto o un cliente sus datos se cargan en el panel de opciones. También se usaron ventanas de confirmación, que detienen el proceso hasta que el usuario responde, y una ventana secundaria para mostrar el ticket.

Trabajar con eventos obligó a cuidar dos aspectos del diseño. Primero, como cada evento puede ocurrir en cualquier momento y con cualquier dato, cada función valida la información antes de actuar. Revisa que no haya campos vacíos, evita que el programa falle si se escribe texto donde se esperaba un número y comprueba que haya existencia del producto y que el pago alcance. Segundo, después de cada evento la interfaz debe quedar actualizada. Para eso, después de cada cambio se vuelven a llenar las listas del catálogo, el inventario, los clientes y el historial. Así, una venta cobrada en una pestaña se refleja de inmediato en el inventario y en el historial. Las listas de productos, clientes y ventas funcionan como la información compartida que conecta todos los eventos.

Los roles se implementaron con la misma idea. Al iniciar sesión, el sistema oculta pestañas o deshabilita botones según el usuario, y con eso limita qué eventos puede generar cada uno. El vendedor no puede abrir el inventario ni eliminar clientes porque esos controles no están disponibles para él.

Cuestionario

1.- ¿Qué paradigmas de programación utiliza python?

Python es un lenguaje **multiparadigma**, es decir, permite programar con distintos estilos:

- **Imperativo y estructurado (procedural):** el programa se escribe como una secuencia de instrucciones organizada con condicionales, ciclos y funciones. Es el paradigma principal de esta práctica.
- **Orientado a objetos:** permite crear clases y objetos con atributos y métodos, herencia y encapsulamiento. En Python todo es un objeto; por ejemplo, cada botón de tkinter es un objeto de la clase tk.Button.
- **Funcional (de forma parcial):** las funciones son objetos que se pueden pasar como parámetros o guardar en variables, y existen lambda, map(), filter() y las comprensiones de listas. En la práctica, al escribir command=cobrar se pasa una función como valor.
- **Orientado a eventos:** con librerías como tkinter, el programa espera acciones del usuario (clics, teclas) y responde ejecutando funciones.

2.- ¿Cuál es la clasificación de los tipos de datos estándar en python?

Categoría	Tipos	Ejemplo	¿Mutable?
Numéricos	int, float, complex	5, 3.14, 2+3j	No
Booleano	bool	True, False	No
Texto	str	"Sofá"	No
Secuencias	list, tuple, range	[1, 2], (1, 2), range(5)	list sí; tuple y range no
Mapeos	dict	{"codigo": 1}	Sí
Conjuntos	set, frozenset	{1, 2, 3}	set sí; frozenset no
Binarios	bytes, bytearray, memoryview	b"abc"	bytearray sí; bytes no
Nulo	NoneType	None	—

Los tipos **mutables** pueden modificarse después de crearse (por ejemplo, agregar elementos a una lista), mientras que los **inmutables** no pueden cambiar; para “modificarlos” se crea un valor nuevo.

3.- ¿Cuáles son las palabras reservadas en Python, (escribe el listado y una breve descripción)?

Las palabras reservadas son palabras que el lenguaje usa para su propia sintaxis y no pueden usarse como nombres de variables o funciones. Python 3 tiene 35 y se pueden consultar con import keyword y print(keyword.kwlist).

Palabra	Descripción	Palabra	Descripción
False	Valor booleano falso.	None	Representa la ausencia de valor.
True	Valor booleano verdadero.	and	Operador lógico “y”.
as	Crea un alias (import ... as, with ... as, except ... as).	assert	Verifica una condición; si es falsa lanza AssertionError.
async	Define una función asíncrona (corrutina).	await	Espera el resultado de una corrutina.
break	Termina un ciclo.	class	Define una clase.
continue	Salta a la siguiente vuelta del ciclo.	def	Define una función.
del	Elimina una variable o un elemento.	elif	Condición adicional dentro de un if.
else	Bloque alternativo de if, for, while o try.	except	Atrapa una excepción.
finally	Bloque de try que siempre se ejecuta.	for	Ciclo que recorre una secuencia.
from	Importa partes específicas de un módulo.	global	Indica que una variable es global dentro de una función.
if	Estructura condicional.	import	Importa un módulo.
in	Pertenencia (x in lista) y recorrido en for.	is	Compara si dos variables son el mismo objeto.
lambda	Crea una función anónima de una línea.	nonlocal	Usa una variable de la función que la contiene.
not	Negación lógica.	or	Operador lógico “o”.
pass	Instrucción vacía; no hace nada.	raise	Lanza una excepción.
return	Devuelve un valor y termina la función.	try	Inicia un bloque que puede generar excepciones.
while	Ciclo que se repite mientras una condición sea verdadera.	with	Administrador de contexto (por ejemplo, abrir y cerrar archivos).
yield	Devuelve valores desde un generador sin terminar la función.		

Desde Python 3.10 también existen las **palabras reservadas suaves** match, case y _, y desde la 3.12 type. Solo funcionan como reservadas en ciertos contextos.

4.- ¿Qué operadores de asignación se pueden utilizar en Python?

Operador	Ejemplo	Equivale a / descripción
=	x = 5	Asigna el valor 5 a x.
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3 (división)
//=	x //= 3	x = x // 3 (división entera)
%=	x %= 3	x = x % 3 (residuo)
**=	x **= 3	x = x ** 3 (potencia)
&=	x &= 3	x = x & 3 (AND a nivel de bits)
|=	x |= 3	x = x | 3 (OR a nivel de bits)
^=	x ^= 3	x = x ^ 3 (XOR a nivel de bits)
>>=	x >>= 3	x = x >> 3 (desplazamiento a la derecha)
<<=	x <<= 3	x = x << 3 (desplazamiento a la izquierda)
@=	a @= b	a = a @ b (multiplicación de matrices, con librerías como NumPy)
:=	if (n := len(lista)) > 0:	Operador “morsa”: asigna un valor dentro de una expresión (Python 3.8+).

También existe la **asignación múltiple** o desempaquetado, que se usó en la práctica: subtotal, iva, total = calcular_totales().

5.- ¿Cómo se realzan los comentarios de línea y multilinea en Python?

Un **comentario de una línea** empieza con el símbolo #; todo lo que sigue en ese renglón es ignorado por el intérprete. Python no tiene una sintaxis especial para **comentarios multilínea**: se escribe un # al inicio de cada renglón, o se usa una cadena entre triples

comillas (""") que no se asigna a ninguna variable. Cuando esa cadena va al inicio de una función, clase o módulo se llama **docstring** y sirve como documentación.

```
# Comentario de una línea
IVA = 0.16  # Comentario al final de una instrucción

# Comentario de varias líneas
# usando un # en cada renglón

"""
Comentario de varias líneas
usando triples comillas
"""
```

6.- ¿Qué es una lista en Python y describe los métodos que utiliza para la lista?

Una **lista** es una colección **ordenada** y **mutable** de elementos, que pueden ser de distintos tipos y repetirse. Se escribe entre corchetes, por ejemplo colores = ["rojo", "azul"], y sus elementos se acceden por índice empezando en 0 (colores[0]). En la práctica, los productos, clientes, ventas y el carrito son listas.

Método	**Descripción**	**Ejemplo**
append(x)	Agrega x al final. (Usado en la práctica)	carrito.append(linea)
extend(iterable)	Agrega todos los elementos de otra colección.	a.extend([4, 5])
insert(i, x)	Inserta x en la posición i.	a.insert(0, "x")
remove(x)	Elimina la primera aparición de x.	a.remove("x")
pop(i)	Quita y devuelve el elemento de la posición i (o el último). (Usado)	carrito.pop(2)
clear()	Vacía la lista. (Usado)	carrito.clear()
index(x)	Devuelve la posición de x.	a.index("x")
count(x)	Cuenta cuántas veces aparece x.	a.count(3)
sort()	Ordena la lista.	a.sort()
reverse()	Invierte el orden.	a.reverse()
copy()	Devuelve una copia de la lista. (Usado)	carrito.copy()

7.- ¿Qué es una tupla y cuáles son los métodos que se utilizan en Python para las tuplas?

Una **tupla** es una colección **ordenada** e **inmutable**: una vez creada no se pueden agregar, quitar ni cambiar sus elementos. Se escribe entre paréntesis, por ejemplo

fuente = ("Arial", 11). Una tupla de un solo elemento necesita una coma: (5,). En la práctica se usaron tuplas para las fuentes, para devolver varios valores en calcular_totales() y al leer curselection(), que regresa una tupla con las filas seleccionadas. Como son inmutables, solo tienen dos métodos:

Método	**Descripción**	**Ejemplo**
count(x)	Cuenta cuántas veces aparece x.	(1, 2, 1).count(1) → 2
index(x)	Devuelve la posición de la primera aparición de x.	("a", "b").index("b") → 1

Además, las tuplas admiten operaciones comunes de secuencias como len(), indexado, rebanadas, in y desempaquetado (a, b = (1, 2)).

8.- ¿Qué son los conjuntos en Python y cuales son los métodos que se utilizan para los conjuntos?

Un **conjunto** (set) es una colección **no ordenada** de elementos **únicos**: no permite duplicados ni acceso por índice. Se escribe entre llaves, por ejemplo tipos = {"Sala", "Comedor"}; un conjunto vacío se crea con set(), porque {} crea un diccionario vacío. Es útil para eliminar duplicados y para operaciones de teoría de conjuntos.

Método	**Descripción**	**Ejemplo**
add(x)	Agrega un elemento.	s.add("Oficina")
update(iterable)	Agrega varios elementos.	s.update([1, 2])
remove(x)	Elimina x; da error si no existe.	s.remove(1)
discard(x)	Elimina x sin dar error si no existe.	s.discard(1)
pop()	Quita y devuelve un elemento cualquiera.	s.pop()
clear()	Vacía el conjunto.	s.clear()
union(otro)	Elementos de ambos conjuntos (|).	a.union(b)
intersection(otro)	Elementos en común (&).	a.intersection(b)
difference(otro)	Elementos de a que no están en b (-).	a.difference(b)
symmetric_difference(otro)	Elementos que están en uno u otro, pero no en ambos (^).	a.symmetric_difference(b)
issubset(otro)	¿Todos sus elementos están en el otro?	a.issubset(b)
issuperset(otro)	¿Contiene todos los elementos del otro?	a.issuperset(b)
isdisjoint(otro)	¿No tienen elementos en común?	a.isdisjoint(b)

Método	Descripción	Ejemplo
copy()	Devuelve una copia.	b = a.copy()

También existen intersection_update(), difference_update() y symmetric_difference_update(), que modifican el conjunto original en lugar de crear uno nuevo.

9.- ¿Qué estructuras de control utilizaste en el desarrollo de la practica?

Estructura	Uso en la práctica
if / else	Validar campos vacíos, cantidades, existencias y pagos; aplicar los permisos según el rol.
if not	Detectar listas o selecciones vacías (por ejemplo, cobrar sin productos o eliminar sin seleccionar).
for	Recorrer productos, clientes, ventas y el carrito para buscar, mostrar, sumar importes y descontar existencias.
try / except	Atrapar el error ValueError al convertir a número lo que escribe el usuario.
return	Salir de una función antes de tiempo cuando un dato no es válido, o devolver un resultado.

Ejemplo de for con if en la búsqueda de un producto:

```
def buscar_producto(codigo):
    for producto in productos:
        if producto["codigo"] == codigo:
            return producto
    return None
```

Ejemplo de try / except al leer el código y la cantidad:

```
try:
    codigo = int(entrada_codigo.get())
    cantidad = int(entrada_cantidad.get())
except ValueError:
    messagebox.showerror("Dato inválido", "Deben ser números enteros.")
    return
```

No se usó un ciclo while explícito: el ciclo que mantiene abierta la ventana lo realiza internamente ventana.mainloop().

10.- Si utilizaste alguna de las funciones de Python, describe el procedimiento realizado para utilizarla

En la práctica se crearon funciones propias y se usaron funciones integradas de Python.

El procedimiento para utilizarlas fue el siguiente:

1. **Definir la función** con def, su nombre, sus parámetros entre paréntesis y dos puntos. El cuerpo se escribe indentado, por ejemplo def buscar_producto(codigo):.
2. **Devolver resultados** con return. Se pueden devolver varios valores separados por comas, que Python agrupa en una tupla, como en calcular_totales().
3. **Llamar a la función** escribiendo su nombre con paréntesis y los argumentos: producto = buscar_producto(6).
4. **Conectarla a un botón** pasando la función **sin paréntesis** en la opción command, por ejemplo command=cobrar. Así tkinter la ejecuta cada vez que se da clic, y no solo una vez al crear el botón.
5. **Conectarla a un evento** con bind(). Por ejemplo, lista_catalogo.bind("<<ListboxSelect>>", seleccionar_del_catalogo); en este caso la función recibe un parámetro evento que tkinter le envía automáticamente.

Funciones integradas y métodos de Python utilizados:

Función o método	**Para qué se usó**
int() / float()	Convertir a número lo que el usuario escribe en las cajas de texto.
str()	Convertir números a texto para mostrarlos en la interfaz.
len()	Obtener el número de ventas (folio) y de clientes.
round()	Redondear el IVA y el total a dos decimales.
.strip()	Quitar espacios al inicio y al final de lo que escribe el usuario.
.center()	Centrar el nombre de la mueblería en el ticket.
.count()	Contar los renglones del ticket para darle altura a la ventana.
.append(), .pop(), .clear(), .copy()	Agregar, quitar, vaciar y copiar elementos de las listas.
.update()	Actualizar los datos de un producto o cliente (diccionario) al editarlo.
datetime.now().strftime()	Obtener la fecha y hora actual con formato día/mes/año hora:minuto para el ticket.

Referencias Bibliográficas

Covatec (2018), Programación en Python-nivel básico, disponible en: https://entrenamiento-python-basico.readthedocs.io/es/latest/index.html

López Ostenero Fernando, García Serrano Ana María (2014), Teoría de los Lenguajes de Programación, Editorial Universitaria Ramón Areces, ISBN: 9788499611396.